

# jev-decision-service 各版本性能横向对比

数据集：AI4I 2020 ( n=2000 ) | 对比版本：0.1.1 / 0.1.3 / 0.1.3+补丁 / 0.1.4 | 真机：NVIDIA DGX Spark ( GB10 ) | 2026-09-27

## 0 结论

把同一个哨兵决策头接到各版本框架上，用同一批 AI4I 测试样本跑到端到端，结果是一条 V 型曲线：0.1.1 表现良好 → 0.1.3 严重回归（自动化率归零）→ 0.1.4 恢复。0.1.4 的价值是「修复回归」，性能本身回到了 0.1.1 水平，而不是超过它；它的真实增量体现在「新增能力」上（OOD 防护、opensource 路径可用）。

|              |                  |                  |            |
|--------------|------------------|------------------|------------|
| 88.8% → 0%   | 0% → 100%        | 0/3 → 3/3        | 5.6%       |
| 0.1.3 自动化率崩盘 | 0.1.4 的 OOD 防护增量 | opensource 路径可用性 | 三版共同的高危漏放率 |

## 1 主表：端到端性能横向对比

数据：AI4I 2020 测试集 n=2000 ( 自然分布，故障 70 条 )。链路：设备状态 → 哨兵决策头 → 校准 → 保护层 → 策略路由 → 最终动作。全部经框架自带的 DecisionEngine 执行 ( warnings 自动合并 )。

| 版本       | 自动化率  | 自动部分准确率 | 高危漏放率 | 正常设备误拦率 | 故障转人工率 | 延迟 p50 (ms) |
|----------|-------|---------|-------|---------|--------|-------------|
| 0.1.1    | 88.8% | 99.66%  | 5.6%  | 8.6%    | 82.9%  | 0.072       |
| 0.1.3    | 0.0%  | —       | 0.0%  | 100.0%  | 100.0% | 0.070       |
| 0.1.3+补丁 | 88.8% | 99.66%  | 5.6%  | 8.6%    | 82.9%  | 0.072       |
| 0.1.4    | 88.8% | 99.66%  | 5.6%  | 8.6%    | 82.9%  | 0.074       |

指标定义：自动化率 = 无需人工即可处置的比例 ( answer\_from\_context / draft\_suggestion )；高危漏放率 = 真值「立即」级但未转人工的比例；正常设备误拦率 = 真值正常却被转人工的比例。

### 1.1 动作分布

| 版本       | 路由动作分布                                                           |
|----------|------------------------------------------------------------------|
| 0.1.1    | answer_from_context 1771 · human_review 223 · draft_suggestion 6 |
| 0.1.3    | human_review 2000 ( 全量 )                                         |
| 0.1.3+补丁 | answer_from_context 1771 · human_review 223 · draft_suggestion 6 |
| 0.1.4    | answer_from_context 1771 · human_review 223 · draft_suggestion 6 |

## 2 能力矩阵：新增防护与可用性

| 能力               | 0.1.1 | 0.1.3      | 0.1.4  | 说明                                  |
|------------------|-------|------------|--------|-------------------------------------|
| OOD ( 分布外输入 ) 拦截 | ✗ 0%  | ✓ 100%     | ✓ 100% | 注入 ood_score=0.9，测 200 条；0.1.1 无此分支 |
| 稀有类命中感知          | ✗     | ✗ ( 过度拦截 ) | ✓      | 0.1.3 一旦声明稀有类就永久拦全部；0.1.4 只在命中时拦    |
| 分布漂移保护           | ✓     | ✓          | ✓      | 三版一致                                |
| 未校准概率降级          | ✓     | ✓          | ✓      | 三版一致                                |
| opensource 路径可用  | ✗ 0/3 | ✗ 0/3      | ✓ 3/3  | 对接 ollama/vLLM 本地模型；真机实测            |

| 能力       | 0.1.1 | 0.1.3 | 0.1.4 | 说明                                                  |
|----------|-------|-------|-------|-----------------------------------------------------|
| 格式错误自动降级 | ✗     | ✗     | ✓     | 0.1.4 新增 <code>_format_error(retryable=True)</code> |
| 自动化率     | 88.8% | 0%    | 88.8% | 0.1.3 因过度拦截归零                                       |

### 3 三个关键结论

#### 3.1 0.1.3 是一次严重回归，不是提升

0.1.3 引入了 OOD 与稀有类保护（这是好事），但稀有类的实现方式有误：只要 profile 里声明了任意样本数不足的类，就无条件拦截所有请求。结果自动化率从 88.8% 直接归零，连健康设备也 100% 转人工。

| 指标      | 0.1.1  | 0.1.3    | 变化       |
|---------|--------|----------|----------|
| 自动化率    | 88.8%  | 0.0%     | -88.8 pt |
| 正常设备误拦率 | 8.6%   | 100.0%   | +91.4 pt |
| 自动部分准确率 | 99.66% | —（无自动处置） | —        |

记忆点：这是一次「加了保护功能反而让系统停摆」的回归。保护逻辑本身是必要的，问题在于触发条件是静态的、与本次输入无关。

#### 3.2 0.1.4 是回归修复，性能持平而非超越

0.1.4 修好了稀有类拦截逻辑后，三项主指标完全回到 0.1.1 水平（自动化率 88.8%、自动准确率 99.66%、误拦率 8.6%）。与 0.1.3+我的补丁 的结果也完全一致——说明两边的修法效果等价。

所以对「迭代有没有带来性能提升」这个问题，诚实的回答是：主指标没有提升，是把 0.1.3 掉的坑填回来了。

#### 3.3 0.1.4 的真实增量在「能力」不在「性能」

| 新增项              | 0.1.1 | 0.1.4 |
|------------------|-------|-------|
| OOD 输入拦截率        | 0%    | 100%  |
| opensource 路径成功率 | 0 / 3 | 3 / 3 |
| 格式错误可降级          | 否     | 是     |
| 稀有类命中感知          | 无     | 有     |

这些能力在本次主表的指标里不会显现——因为测试用的是结构化决策头路径，且未注入 OOD 分数。但它们对实际部署价值很大：OOD 防护意味着传感器异常、设备型号变更时系统不会硬用错误模型输出。

### 4 一个三版共同的现象：5.6% 高危漏放

所有版本的高危漏放率都是 5.6%（8 / 142），与框架版本无关。原因在模型层而非框架层：evaluator 依据决策头的 severity 判断是否转人工，当决策头把某个真值「立即」的样本预测成较低档位时，该样本就被自动处置了。

这是决策头在 severity 任务上的识别误差（该头准确率约 98.5%），要降下来只能从数据侧入手（补少数类样本、重新标注 severity）。

### 5 测试方法与公平性说明

| 要素  | 设定                                         |
|-----|--------------------------------------------|
| 数据集 | AI4I 2020 测试集 n=2000（自然分布，故障 70 条），各版本完全相同 |

|      |                                              |
|------|----------------------------------------------|
| 要素   | 设定                                           |
| 决策头  | 同一个哨兵决策头（固定种子 20260921、同数据训练、同温度校准）          |
| 问题集  | 同一套工业维护版 QuestionProfile（沿用 policy 认识的问题 id） |
| 执行链路 | 框架自带 DecisionEngine（warnings 自动合并，含记忆与编排层）   |
| 策略配置 | PolicyConfig(urgent_score=3.0)；只有「立即」才强制转人工  |
| 指标   | 自动化率 / 自动部分准确率 / 高危漏放率 / 正常误拦率 / 故障转人工率 / 延迟 |

说明：ev 的默认「模型」是规则引擎（对话路由用），规则词表跨版本几乎未变，因此单看规则引擎没有可比性。本次对比固定决策头、只让框架版本变化，测量的正是「框架层」对同一批决策的加工质量——这也是各版本差异的真正来源。

## 6 建议

| # | 建议                                                           |
|---|--------------------------------------------------------------|
| 1 | 把「0.1.3 自动化率归零」写进回归测试：一旦某版本自动化率跌破阈值即报警。这类回归不报错、不崩溃，只能靠指标监控发现 |
| 2 | 稀有类、OOD 这类保护逻辑，触发条件必须与本次输入/本次预测相关，不能是静态声明。建议在 profile 层加一条自检 |
| 3 | 0.1.4 可作为后续开发的基线；主指标已回到 0.1.1 水平，且多了 OOD 与 opensource 能力     |
| 4 | 下一步优先做「哨兵决策头正式接入 StructuredHeadProvider」——框架已稳，接上即完整系统       |

复现：python3 version\_bench.py <版本目录> <输出json> | OOD 补充测试：ood\_bench.py | 全部数据来自 gx10-902c 节点实测